

Reviewer Pack — Algebraic Shifting Facet Count Bounds Tseitin Resolution Len...

Entry 312ea3700c09 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 04:56:00 UTC

Algebraic Shifting Facet Count Bounds Tseitin Resolution Length

Entry ID: 312ea3700c09 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 04:56:00 UTC

1. Conjecture

Field A (mathematical branch): Algebraic shifting **Field B** (complexity object): Resolution length of Tseitin formulas

Statement:

For a Tseitin formula on a graph G , the resolution length is $\geq 2^{\{\Omega(\nu(G))\}}$, where $\nu(G)$ is the number of facets in the algebraic shifted complex of G . This holds for all graphs with expansion parameter $\lambda(G) \geq 1/\sqrt{n}$.

Rationale (proposer’s reasoning):

Algebraic shifting captures combinatorial structure of graphs via simplicial complexes, while resolution length measures proof complexity. The conjecture links structural properties of shifted complexes to proof complexity, leveraging the interplay between algebraic topology and proof complexity.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2eb2e54045f300f1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_graph(n):
        edges = set()
        for i in range(n):
            for j in range(i + 1, n):
                if random.random() < 0.5:
                    edges.add((i, j))
        return edges

    def lex_order_complex(edges, n):
        vertices = list(range(n))
        facets = []

        def dfs(path, start):
            if len(path) == n:
                facets.append(tuple(sorted(path)))
                return
            for v in range(start, n):
                if (path[-1], v) not in edges and (v, path[-1]) not in edges:
                    dfs(path + [v], v)

        for i in range(n):
            dfs([i], i)
        return facets

    def resolution_length(facets):
        clauses = []
        for facet in facets:
            clause = [-x - 1 for x in facet]
            clauses.append(clause)

    def dpll(clauses, assignment, unit_clause=None):
        if not clauses:
            return True
```

```

    if unit_clause is not None:
        var = unit_clause[0]
        if var > 0:
            assignment[var] = True
        else:
            assignment[-var] = False
        clauses = [c for c in clauses if var not in c and -var not in c]

    literal, polarity = next((l, p) for l, c in enumerate(clauses) for p in (True, False))
    new_assignment = assignment.copy()
    new_assignment[literal] = polarity
    if dpll(clauses, new_assignment):
        return True

    new_assignment[literal] = not polarity
    if dpll(clauses, new_assignment):
        return True

    return False

assignment = {}
return len(facets) if dpll(clauses, assignment) else 0

n = random.randint(5, 40)
while True:
    graph = generate_graph(n)
    expansion_param = sum(len(list(graph.neighbors(v))) for v in range(n)) / (n * (n - 1))
    if expansion_param >= 1 / math.sqrt(n):
        break

facets = lex_order_complex(graph, n)
resolution_len = resolution_length(facets)

c = 0.3
conjecture_holds = resolution_len >= 2 ** (c * len(facets))
counterexample = "" if conjecture_holds else f"Resolution length {resolution_len} < {2 ** (c * len(facets))}"

return {
    "metric_name": "Resolution Length",
    "metric_value": resolution_len,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)

```

```

std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}\"")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_91e2d45b.py", line 105, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_91e2d45b.py", line 80, in run_trial
    expansion_param = sum(len(list(graph.neighbors(v))) for v in range(n)) / (n * (n - 1))
                        ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_91e2d45b.py", line 80, in <genexpr>
    expansion_param = sum(len(list(graph.neighbors(v))) for v in range(n)) / (n * (n - 1))
                        ~~~~~
AttributeError: 'set' object has no attribute 'neighbors'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to AttributeError in graph.neighbors() call on set objects, preventing data collection | next: Fix graph representation to use a structure with neighbors() method (e.g., NetworkX Graph) and re-run experiments

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	135901
2	propose	ollama_remote	qwen3:8b	0	0	55717
3	propose	ollama_remote	qwen3:8b	0	0	138395

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	propose	ollama_remote	qwen3:8b	0	0	97423
5	preregistration	ollama_remote	qwen3:8b	0	0	32068
6	novelty	ollama_remote	qwen3:8b	0	0	27255
7	novelty	ollama_remote	qwen3:8b	0	0	25794
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14635
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	36165
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17999
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13606
12	judge	ollama_remote	qwen3:8b	0	0	114671

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 709629 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/312ea3700c09.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/312ea3700c09.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/312ea3700c09.tar.gz` (if generated)